

POO 3 (JAVA ET JAVA AVANCÉ)

Prof. Nisrine DAD

4° Ingénierie Informatique et Réseaux - Semestre I

Ecole Marocaine des Sciences d'Ingénieur

Année Universitaire : 2024/2025

III. CONCEPTS DE LA PROGRAMMATION ORIENTÉE OBJETS (POO) EN JAVA

III. CONCEPTS DE LA POO EN JAVA

Classes abstraites: Concept

- Une **classe abstraite** est une classe qui **ne permet pas l'instanciation** des objets de son type.
- C'est une classe incomplète puisqu'elle contient certaines méthodes dont leurs corps sont définies dans la classe dérivée. En effet, **elle doit être héritée**.
- Une classe abstraite comporte **une ou plusieurs méthodes abstraites**, et elle **peut comporter d'autres méthodes non abstraites**. Une méthode abstraite est une méthode qui ne possède pas d'instructions (corps). Elle comporte **seulement une signature** de définition **suivie du caractère point-virgule**.
- Une méthode abstraite doit être précédée par le mot clé **abstract**.
- **Exemple :** `public abstract double fAbstraite();`

III. CONCEPTS DE LA POO EN JAVA

Classes abstraites: Syntaxe

```
public abstract class Personne {  
  
    public void affiche() {  
        System.out.println("Personne");  
    } // affiche() est définie dans Personne  
  
    public abstract void determineAge (int annee) ;  
    //determineAge() est une méthode abstraite,  
    //elle n'est pas définie dans Personne.  
}
```

```
Personne p ;  
//on peut déclarer une référence sur un objet  
//de type Personne ou dérivé.  
p = new Personne (); //Erreur, pas  
//d'instanciation d'objets d'une classe  
//abstraite.
```

III. CONCEPTS DE LA POO EN JAVA

Classes abstraites: Règles

- Une classe est automatiquement abstraite (déclarée avec le mot clé ***abstract***) si une de ses méthodes est abstraite. Une méthode abstraite **ne peut pas être privée**, ce qui est logique puisque son corps sera défini dans une classe dérivée.
- Les **noms d'arguments muets** doivent figurer dans la définition d'une méthode abstraite.

```
public abstract void determineAge (int); //Erreur : nom argument fictif est obligatoire
```

```
public abstract void determineAge (int annee); //Correct
```

- Une classe dérivée d'une classe abstraite **n'est pas obligée de redéfinir** toutes les méthodes abstraites, elle peut ne redéfinir aucune, mais **elle reste abstraite** tant qu'il y a encore des méthodes abstraites non implémentées.
- Une classe dérivée d'une classe **non abstraite** peut être déclarée **abstraite**.

III. CONCEPTS DE LA POO EN JAVA

Classes abstraites: Règles

- Une classe abstraite est **non instanciable** (donc il n'est pas possible d'utiliser les constructeurs de cette classe et donc on ne peut pas créer des objet de son type.)
- Une classe dérivée d'une classe abstraite ne devient **concrète** que si elle implémente toutes les méthodes abstraites de sa classe de base.
- Une méthode abstraite ne possède pas de corps, mais doit être implémentée dans les classes dérivées non abstraites : abstract fonction (arguments); ce n'est qu'une signature .
- Une classe abstraite peut contenir **des méthodes non abstraites** et **des déclarations de variables ordinaires**.
- Une classe abstraite **n'est pas obligée de contenir des méthodes abstraites**.

III. CONCEPTS DE LA POO EN JAVA

Classes abstraites: Intérêt

- Une classe abstraite peut être **utile** pour **imposer** l'existence de certaines **méthodes** dans des classes, **sans savoir comment elles seront implémentées**.

```
public abstract class Personne {  
    public abstract void affiche();  
}
```

```
public class Enseignant extends Personne{  
    public void affiche() {  
        System.out.println("Je suis un  
            + "enseignant.");  
    }  
}
```

```
public class Etudiant extends Personne{  
    public void affiche() {  
        System.out.println("Je suis  
            + "un étudiant.");  
    }  
}
```

- Une classe qui **hérite** d'une classe abstraite **doit redéfinir les méthodes abstraites**, sinon elle est abstraite également.

III. CONCEPTS DE LA POO EN JAVA

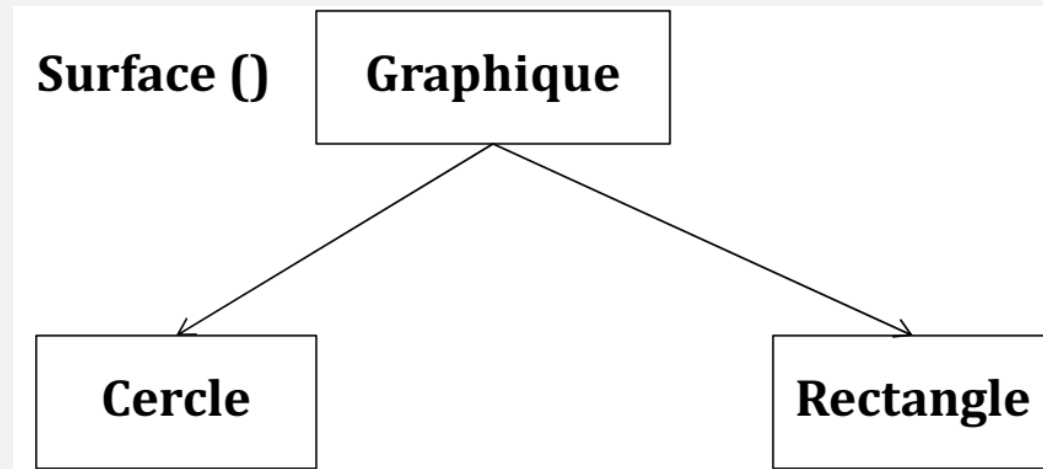
Classes abstraites: Intérêt

- Les classes abstraites facilitent largement la conception orientée objet.
- On peut placer dans une classe abstraite toutes les fonctionnalités dont on souhaite disposer pour les classes descendantes :
 - Soit **sous la forme d'une implémentation complète de méthodes** (non abstraites) et **de champs** lorsqu'ils sont communs à tous les descendants,
 - Soit **sous forme d'interface des méthodes abstraites** dont on est alors sûr qu'elles existeront dans toute classe dérivée instanciable.

III. CONCEPTS DE LA POO EN JAVA

Classes abstraites: Exemple

- Considérons la hiérarchie des classes suivantes :



- On veut que les classes **Cercle** et **Rectangle** disposent de la méthode `surface()` retournant la surface de la figure géométrique définie par la classe.

III. CONCEPTS DE LA POO EN JAVA

Classes abstraites: Exemple

```
public abstract class Graphique {  
    private int x, y; //Voilà les champs à hériter  
  
    //Une classe abstraite ne peut pas être instanciée, mais elle peut avoir  
    //un constructeur :  
    public Graphique ( int x, int y) {  
        this.x = x ; this.y = y ;}  
  
    //Voilà la méthode abstraite à compléter :  
    public abstract double surface();  
  
    //Il peut y avoir des méthodes non abstraites  
    public void affiche() {  
        System.out.println("Le centre de l'objet se trouve dans : " + x + " et " + y);  
        System.out.println("La surface de l'objet est : " + this.surface());  
    }  
}
```

III. CONCEPTS DE LA POO EN JAVA

Classes abstraites: Exemple

```
public class Cercle extends Graphique {  
    private double rayon;  
  
    public Cercle(int x, int y, double r) {  
        super(x, y);  
        rayon = r;}  
  
    public double surface() {  
        return (2 * 3.14 * rayon);}  
}
```

```
public class Rectangle extends Graphique {  
    private int largeur, longueur;  
  
    public Rectangle(int x, int y, int la, int lo){  
        super(x, y);  
        this.longueur = lo;  
        this.largeur = la;}  
  
    public double surface() {  
        return (longueur * largeur);}  
}
```

III. CONCEPTS DE LA POO EN JAVA

Classes abstraites: Exemple

```
public class TestGraphique {  
    public static void main(String[] args) {  
        Graphique[] tab = new Graphique[6];  
        tab[0] = new Graphique (3,2);  
        tab[1] = new Cercle(10, 7, 3);  
        tab[2] = new Rectangle(4, 7, 8, 6);  
        tab[3] = new Rectangle(8, 10, 12, 10);  
        tab[4] = new Cercle(8, 5, 3);  
        tab[5] = new Rectangle(10, 17, 3, 8);  
        for (int i = 1; i <= 5; i++) {  
            tab[i].affiche();  
            // Y-a-t-il des erreurs?  
            // Quel est le résultat?  
        }  
    }  
}
```

`tab[0] = new Graphique (3,2);`
Erreur, classe abstraite est non instanciable.

Il est tout de même possible de déclarer des variables de ce type qui pourront recevoir des objets créés à partir des sous-classes concrètes

Le centre de l'objet se trouve dans :10 et 7
La surface de l'objet est :18.84
Le centre de l'objet se trouve dans :4 et 7
La surface de l'objet est :48.0
Le centre de l'objet se trouve dans :8 et 10
La surface de l'objet est :120.0
Le centre de l'objet se trouve dans :8 et 5
La surface de l'objet est :18.84
Le centre de l'objet se trouve dans :10 et 17
La surface de l'objet est :24.0

III. CONCEPTS DE LA POO EN JAVA

Interfaces: Concept

- Une **interface** est un ensemble d'opérations utilisées pour spécifier un **service offert** par une classe.
- Elle peut être vue comme une classe **sans attributs (sauf les constantes)** et dont toutes les opérations sont spécifiées mais **pas définies** à priori.
- Elle peut être vue comme **un contrat** ou **un modèle** que doivent offrir toutes les classes qui se réclament (implémentent) de cette interface.

```
public interface Polygone {  
    int surface();  
    void deplace(int x, int y);  
}
```

III. CONCEPTS DE LA POO EN JAVA

Interfaces vs classes abstraites

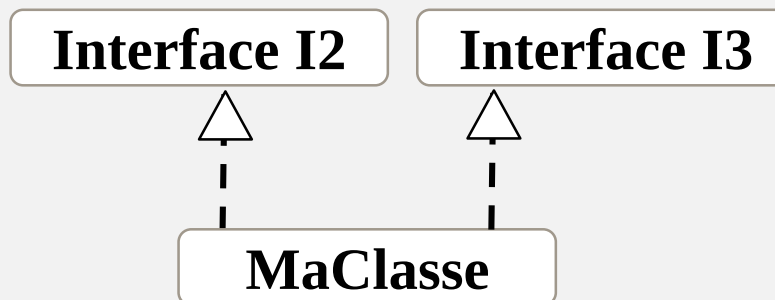
- Le corps d'une interface ressemble beaucoup à celui d'une classe abstraite avec cependant les différences suivantes :
- Une interface ne peut définir que des méthodes abstraites. Le modificateur **abstract n'est pas nécessaire**.
- Toutes les méthodes abstraites sont **implicitement publiques** même si le modificateur public est omis.
- Une interface **ne peut pas déclarer des méthodes statiques comme abstraites**.
- Une interface peut déclarer des **méthodes par défaut** : avec une implémentation par défaut, optionnelle.
- Une interface **ne peut pas définir de champs d'instance**.
- Une interface peut cependant contenir **des constantes déclarées static et final** (tous les champs sont **implicitement static et final** même si les modificateurs correspondants sont omis).
- Une interface **ne définit pas de constructeur** (on ne peut pas l'instancier et elle n'intervient pas dans le chaînage des constructeurs).

III. CONCEPTS DE LA POO EN JAVA

Interfaces vs classes abstraites

- Si l'on considère une classe abstraite n'implémentant aucune méthode et aucun champ (sauf les constantes), on aboutit à la notion d'interface.
- **L'interface est plus riche** qu'un simple cas particulier des classes abstraites :
- **Une classe peut implémenter plusieurs interfaces** (alors qu'une classe ne pouvait dériver que d'une seule classe abstraite).

```
public class MaClasse implements I1, I2{}
```



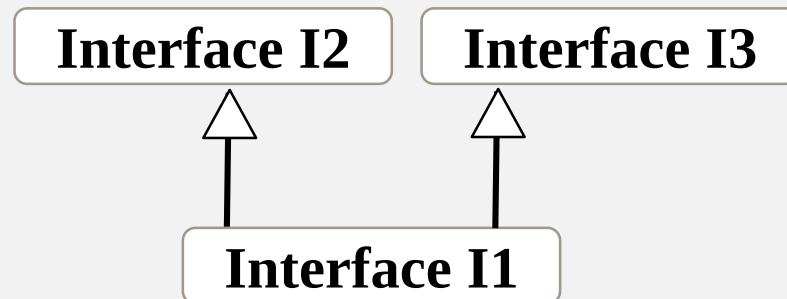
```
public class MaClasse extends C1, C2{} // Pas possible
```

III. CONCEPTS DE LA POO EN JAVA

Interfaces vs classes abstraites

- **L'interface est plus riche** qu'un simple cas particulier des classes abstraites :
 - Les interfaces peuvent se dériver et même utiliser l'héritage multiple:

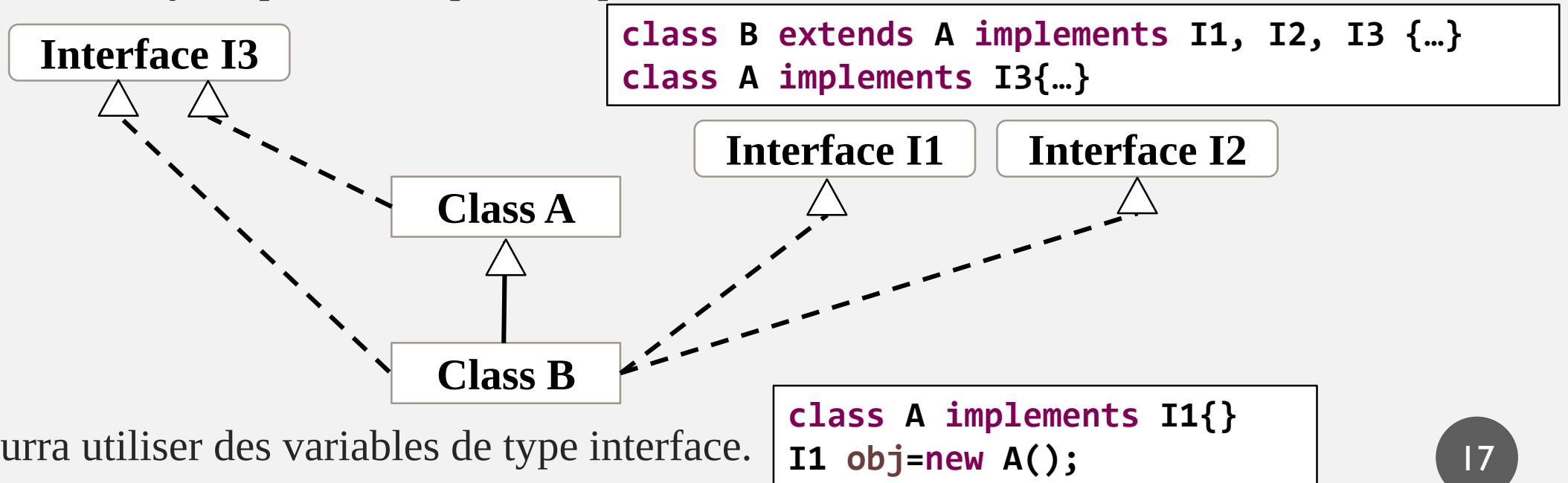
```
public interface I1 extends I2, I3{.....}
```



III. CONCEPTS DE LA POO EN JAVA

Interfaces vs classes abstraites

- **L'interface est plus riche** qu'un simple cas particulier des classes abstraites :
 - Une classe dérivée peut implémenter 1 ou plusieurs interfaces, elle peut même ré-implémenter une interface déjà implémentée par la superclasse.



- On pourra utiliser des variables de type interface.

III. CONCEPTS DE LA POO EN JAVA

```
public interface Polygone {  
    int max = 100;  
  
    int surface();  
    void deplace(int x, int y);}
```



```
public interface Polygone {  
    public static final int max = 100;  
  
    public abstract int surface();  
    public abstract void deplace(int x, int y);}
```

Une interface peut renfermer aussi des constantes

```
public class Cercle implements Affichable, Polygone {  
    private double rayon;  
    private Point centre;  
  
    public double surface() {  
        return (double) (2 * 3.14 * rayon);  
    }  
    public void deplace(int x, int y) {  
        centre.setX(centre.getX()+x);  
        centre.setY(centre.getY()+y);  
    }  
    public void afficher() {  
        System.out.println("Je suis un cercle");  
    }  
}
```

```
public interface Affichable {  
    void afficher();  
}
```

Lorsqu'on définit une classe, on peut préciser qu'elle implémente une ou plusieurs interface en utilisant le mot clé « *implements* »

III. CONCEPTS DE LA POO EN JAVA

Interfaces et dérivation

- L'interface est totalement indépendante de l'héritage : Une classe dérivée peut implémenter une ou plusieurs interfaces.
- L'implémentation d'une ou plusieurs interfaces (implements) peut être combinée avec l'héritage simple (extends).
- La clause implements doit suivre la clause extends.

```
public class Cercle extends Graphique implements Affichable, Polygone
```

III. CONCEPTS DE LA POO EN JAVA

Interfaces: Caractéristiques

- Lorsqu'une classe déclare une interface dans sa clause implements, elle indique ainsi qu'elle s'engage à fournir une implémentation (c'est-à-dire un corps) pour chacune des méthodes de cette interface.
- Si une classe implémente une interface mais ne fournit pas d'implémentation pour toutes les méthodes de l'interface, c-à-d elle hérite des méthodes (abstraites) non implémentées , elle doit être déclarée abstract.
- Si une classe implémente plus d'une interface, elle doit implémenter toutes les méthodes de chacune des interfaces mentionnées dans la clause implements (ou alors doit être déclarée abstract).

III. CONCEPTS DE LA POO EN JAVA

```
public interface Polygone {  
    int max = 100;  
  
    int surface();  
    void deplace(int x, int y);}
```

```
public interface Affichable {  
    void afficher();  
}
```

La classe Carre est déclarée abstract car elle n'implémente pas toutes les méthodes des interfaces implémentée

Toutes les méthodes abstraites sont implicitement publique: Pas besoin du mot clé *public*

```
public abstract class Carre implements Affichable, Polygone{  
  
    abstract double surface();  
    abstract void deplace(int x, int y) ;  
  
    public void afficher() {  
        System.out.println("Je suis un carré.");  
    }  
}
```

III. CONCEPTS DE LA POO EN JAVA

Interfaces: Conflits des noms

```
interface I1 {  
    void f(int n) ;  
    void g();  
}
```

```
interface I2 {  
    void f(float x);  
    void g();  
}
```

```
class A implements I1, I2{  
    void f(int n) {}  
    void f(float x) {}  
    void g();  
}
```

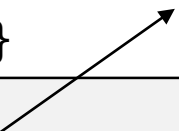


A doit implémenter la méthode g() existante dans les 2 interfaces une seule fois

```
interface I1 {  
    void f(int n) ;  
    void g();  
}
```



```
interface I2 {  
    void f(float x);  
    int g();  
}
```



la méthode « g () » est présentée de 2 façons différentes dans I1 et I2

```
class A implements I1, I2{  
    void f(int n) {}  
    void f(float x) {}  
    void g();  
    int g();  
}
```



Erreur car void g() et int g() ne peuvent pas coexister au sein de la même classe

III. CONCEPTS DE LA POO EN JAVA

Méthodes génériques

- Les méthodes génériques sont celles qui sont écrites avec une seule déclaration et qui peuvent être appelées avec des arguments de différents types.
- En fonction des types d'arguments passés à la méthode générique, le compilateur gère chaque appel de méthode de manière appropriée.

```
public static <E> void genericDisplay(E element){  
    System.out.println(element.getClass().getName()  
        + "=" + element);  
}
```

```
String str = "Type String";  
Carre.genericDisplay(str);
```

```
Integer i = 25;  
Carre.genericDisplay(i);
```

```
java.lang.String=Type String  
java.lang.Integer=25
```

III. CONCEPTS DE LA POO EN JAVA

Méthodes génériques

- Voici les règles pour définir les méthodes génériques:
 - Les méthodes génériques ont **un type paramétré** (qui représente un paramètre de **type générique**) **avant le type de retour** de la déclaration de méthode.

```
public static <E> void genericDisplay(E element){  
    System.out.println(element.getClass().getName()  
        + "=" + element);}
```

- Les méthodes génériques **peuvent avoir différents types paramétrés séparés par des virgules** dans la signature de méthode.

```
public static <E,T> void affichage2(E element1, T element2){  
    System.out.println("element 1= " + element1 + ", element2=  
        " + element2);}
```

```
affichage2(str,i);
```

```
element 1= Type String, element2= 25
```


III. CONCEPTS DE LA POO EN JAVA

Méthodes génériques

Type paramétré

- Les conventions de dénomination de types paramétrés sont importantes pour apprendre la généricité en Java. Ils doivent contenir une seule lettre en majuscule. Les types paramétrés courants sont les suivants: **T – Type**, **E – Element**, **K – Key**, **N – Number**, **V – Value**.
- Le type T, par exemple, indique que la méthode peut faire référence à n'importe quel type objet (comme String, Integer et Personne) et non pas un type de base.
- On peut utiliser des types spécifiques héritant d'une classe spécifique. Exemple:

<N extends Number>, < T extends ClassA >

III. CONCEPTS DE LA POO EN JAVA

```
public class Main {  
    // méthode générique  
    public static <E> void display(E[] tab) {  
        for(E e : tab) {  
            System.out.printf("%s ", e);  
        }  
        System.out.println();  
    }  
    public static void main(String args[]) {  
        Integer[] i = {1, 2, 3};  
        System.out.println("Tableau des entiers:");  
        display(i);    // passer un tableau entier  
        Character[] c = {'A', 'B', 'C'};  
        System.out.println("\nTableau des caractères:");  
        display(c);    // passer un tableau de caractères  
    }  
}
```

Tableau des entiers:

1 2 3

Tableau des caractères:

A B C

III. CONCEPTS DE LA POO EN JAVA

Classes génériques

- La déclaration d'une classe générique ressemble à une déclaration de classe non générique, sauf que le nom de la classe est suivi du type paramétré. Exemple:

```
class MaClasse <T> {}
```

- Comme pour les méthodes génériques, la section des paramètres d'une classe générique peut avoir un ou plusieurs paramètres de type séparés par des virgules.

```
class MaClasse <T,E> {}
```

- Ces classes sont appelées classes paramétrées ou types paramétrés car elles acceptent un ou plusieurs paramètres.
- Une classe qui peut faire référence à n'importe quel type est appelée une classe générique.

III. CONCEPTS DE LA POO EN JAVA

Classes génériques

- Exemple:

```
class MaClasse <T> {  
    T obj;  
  
    void setObj(T obj){  
        this.obj = obj;  
    }  
  
    T getObj(){  
        return obj;  
    }  
}
```

```
public class TestGen {  
  
    public static void main(String args[]) {  
        MaClasse<Integer> obj = new MaClasse<Integer>();  
        obj.setObj(5);  
        //obj.setObj("String"); // Erreur de compilation  
        System.out.println(obj.getObj());  
    }  
}
```

5

III. CONCEPTS DE LA POO EN JAVA

Interfaces génériques

- La généricité fonctionnent également avec les interfaces.
- Les interfaces génériques sont spécifiées tout comme les classes génériques.

```
interface MyInterface<T> { ... }
```

- Si une classe implémente une interface générique, on doit indiquer les types des paramètres avec lesquels la classe fonctionne.

```
class MyClass implements MyInterface<Integer> {...} //On a spécifié la param à Integer;
```

- Sinon, cette classe est aussi générique.

```
class MyClass<T> implements MyInterface<T> {...}
```

III. CONCEPTS DE LA POO EN JAVA

L'interface Comparable

- **Comparable** est une interface générique définie dans l'API Java (package **java.lang**). Elle impose aux classes qui vont l'implémenter la définition de la méthode `compareTo()`:

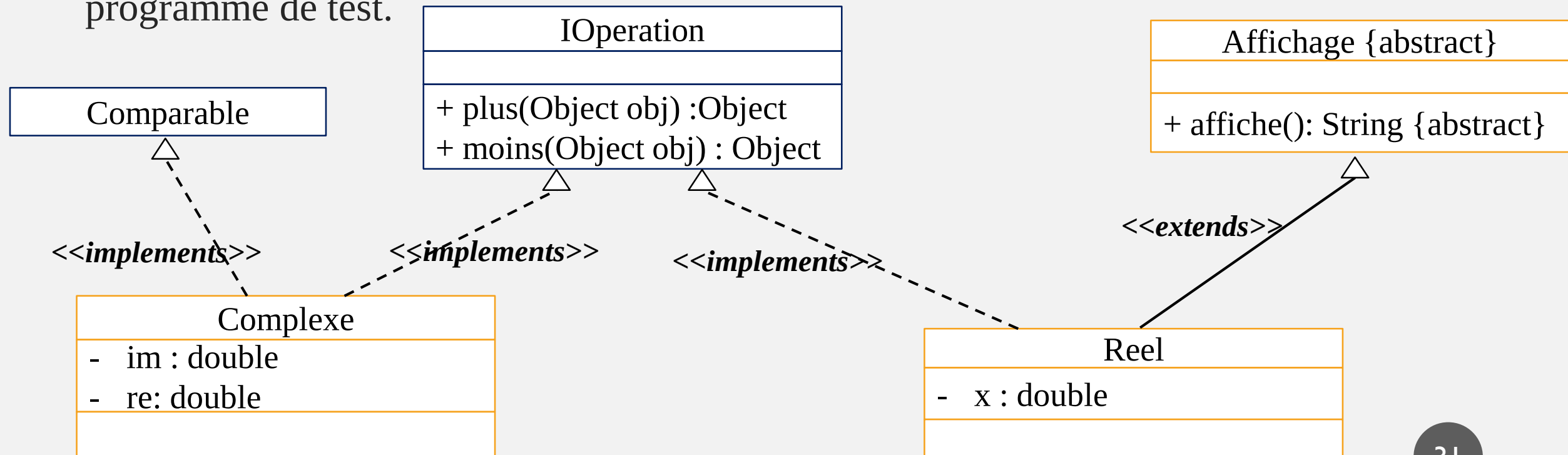
```
public interface Comparable<T> {  
    public int compareTo(T o);  
}
```

- La méthode ***compareTo()*** permet de comparer l'objet courant (`this`) à l'objet `o` reçu en argument et de renvoyer un entier (dont la valeur exacte est sans importance) :
 - **négatif** (la valeur -1) si l'on considère que l'objet courant est « **inférieur** » à l'objet `o` (au sens de l'ordre qu'on veut définir),
 - **nul** (la valeur 0) si l'on considère que l'objet courant **est égal** à l'objet `o` (il n'est ni inférieur, ni supérieur),
 - **positif** (la valeur 1) si l'on considère que l'objet courant est « **supérieur** » à l'objet `o`

III. CONCEPTS DE LA POO EN JAVA

Exercice

- Implémentez le code correspondant au diagramme de classes suivant et donnez un programme de test.



III. CONCEPTS DE LA POO EN JAVA

Exercice: Corrigé

```
public class Reel extends Affichage implements IOperation<Reel>{
    private double x;

    public Reel(double x) {
        this.x = x;}

    public String affiche() {
        return Double.toString(x);}

    public Reel plus(Reel r1) {
        Reel resultat= new Reel(this.x+r1.x);
        return resultat;}

    public Reel moins(Reel r1) {
        Reel resultat= new Reel(this.x-r1.x);
        return resultat;}}
```

```
public interface IOperation <T>{
    public T plus(T nombre);
    public T moins(T nombre);
}
```

```
public abstract class Affichage {
    public abstract String affiche();
}
```


III. CONCEPTS DE LA POO EN JAVA

```
public class Complexe implements IOperation<Complexe>, Comparable<Complexe>{
```

```
    private double im, re;
```

```
    public Complexe(double im, double re) {  
        this.im = im;  
        this.re = re;}  
}
```

```
    public Complexe plus(Complexe nombre) {  
        Complexe resultat=new Complexe(this.im+nombre.im, this.re+nombre.re);  
        return resultat;}  
}
```

```
    public Complexe moins(Complexe nombre) {  
        Complexe resultat=new Complexe(this.im-nombre.im, this.re-nombre.re);  
        return resultat;}  
}
```

```
    public String toString() {  
        return "Complexe [im=" + im + ", re=" + re + "];"  
    }  
}
```

```
    public int compareTo(Complexe o) {  
        if(this.module()==o.module())  
            return 0;  
        if(this.module()<o.module())  
            return -1;  
        else  
            return 1;}  
}
```

```
    public double module() {  
        return Math.sqrt(im*im+re*re);}  
}
```

III. CONCEPTS DE LA POO EN JAVA

```
Complexe c1=new Complexe(1,2);
Complexe c2=new Complexe(2,3);

System.out.println(c1);
System.out.println(c2);

Reel r1=new Reel(1);
Reel r2=new Reel(2);

System.out.println(r1.affiche());
System.out.println(r2.affiche());

System.out.println(c1.plus(c2));

System.out.println(r1.moins(r2).affiche());
```

```
int a=c1.compareTo(c2);
switch(a) {
    case -1: System.out.println(c1.toString()+"
est inférieur à "+c2.toString());
    break;
    case 0: System.out.println(c1.toString()+"
est égal à "+c2.toString());
    break;
    case 1: System.out.println(c1.toString()+"
est supérieur à "+c2.toString());
    break;
}
```

```
Complexe [im=1.0, re=2.0]
Complexe [im=2.0, re=3.0]
1.0
2.0
Complexe [im=3.0, re=5.0]
-1.0
```

```
Complexe [im=1.0, re=2.0] est inférieur à
Complexe [im=2.0, re=3.0]
```